



Instituto Tecnológico Superior de Lerdo

Actividad de evaluación – ABPj

Administración y Organización de Datos

Alumnos:

Lucia Jacqueline Delgado Martinez -242310882

Jacqueline Sanchez Gonzalez-242310048

Kevin Daniel Esparza Ojeda-242310204

MTI. Jesus Salas Marin

Carrera: Ingenieria en Informatica

Fecha: 20/04/2026

Cuarto semestre

Sección: "A"

Descripción de el tema del programa

Nuestro sistema está implementado en una planta industrial dedicada al almacenamiento de sustancias químicas peligrosas. En este entorno, es fundamental controlar estrictamente el acceso a ciertas áreas para prevenir accidentes, robos o manipulaciones indebidas. Debido al alto riesgo que representan estos materiales, solo el personal autorizado puede ingresar a zonas críticas, garantizando así la seguridad de los trabajadores, las instalaciones y el medio ambiente.

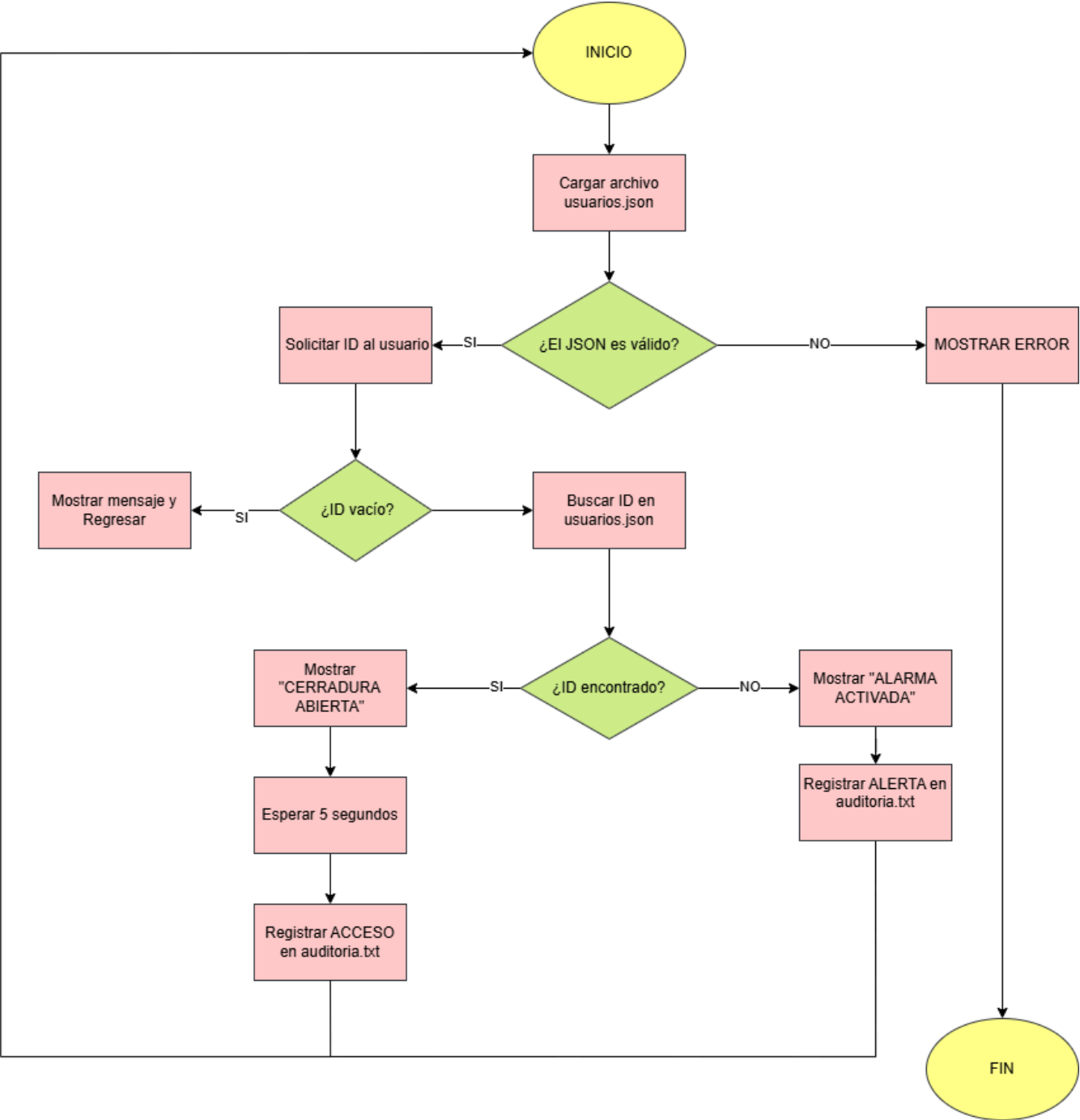
Introducción

El programa desarrollado consiste en un sistema de control de acceso que permite validar la entrada de usuarios mediante un identificador único. Utiliza un archivo JSON para almacenar la información de los empleados, un script en Python para procesar las solicitudes de acceso y un archivo TXT para registrar cada intento realizado. Además, cuenta con una interfaz en PHP que muestra los registros de forma organizada y visual. Este sistema simula el funcionamiento de un control de seguridad en tiempo real, permitiendo distinguir entre accesos permitidos y denegados de manera clara y eficiente.

¿Por qué usar archivos planos?

En este tipo de entorno industrial, no siempre es viable usar bases de datos complejas debido a limitaciones de hardware o conectividad. Por eso usamos archivos planos como JSON y TXT, ya que permiten un acceso rápido, son fáciles de implementar en dispositivos como Raspberry Pi y reducen el riesgo de fallos del sistema.

DIAGRAMA



control.py++

```
control.py > ...  
1 import json  
2 from datetime import datetime
```

json = permite leer el archivo usuarios.json como si fuera un diccionario

datetime = se usa para obtener la fecha y hora actual

```
3  
4 def cargar_usuarios():  
5     """
```

Función = cargar_usuarios()

Carga los datos del archivo usuarios.json usando la librería json y devuelve la lista de usuarios. Incluye manejo de errores en caso de que el archivo no exista o esté mal formado.

```
5     try:  
6         with open("usuarios.json", "r") as archivo:  
7             datos = json.load(archivo)  
8             return datos["usuarios"]
```

with open = abre el archivo en modo lectura ('r')

datos = convierte el JSON en un diccionario de Python

return = devuelve la lista de usuarios

```
9     except json.JSONDecodeError:  
10         print("ERROR: El archivo JSON está mal formado")  
11         return []  
12     except FileNotFoundError:  
13         print("ERROR: Archivo usuarios.json no encontrado")  
14         return []
```

except json = detecta si el JSON está mal escrito

except fileNotFoundError = detecta si el archivo no existe

- muestra error
- retorna lista vacía para evitar fallos

```
16 v def registrar_log(mensaje):
17 v     with open("auditoria.txt", "a") as log:
18         fecha = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
19         log.write(f"{fecha} - {mensaje}\n")
```

Función = registrar_log()

Guarda un registro en el archivo auditoria.txt con la fecha y hora actual junto con el mensaje del intento de acceso. Usa el modo 'a' para no borrar información previa.

with open =

- abre en modo append ('a')
- no borra datos anteriores

fecha = obtiene fecha y hora actual

log.write = guarda el registro en una nueva línea

```
21 def buscar_usuario(id_ingresado, usuarios):
22     for usuario in usuarios:
23         if usuario["id_tarjeta"] == id_ingresado:
24             return usuario
25     return None
```

Función = buscar_usuario()

Recorre la lista de usuarios y verifica si el ID ingresado coincide con alguno existente. Si lo encuentra devuelve el usuario si no devuelve none.

for usuario in usuarios = recorre la lista de usuarios

if usuario["id_tarjeta"] == id_ingresado = compara el ID ingresado

si lo encuentra:

return usuario

si no:

return None

```

27  def main():
28      usuarios = cargar_usuarios()
29
30      id_ingresado = input("Ingrese ID de tarjeta: ")
31
32      usuario = buscar_usuario(id_ingresado, usuarios)

```

Función principal = main()

Es la función principal que coordina todo el proceso: carga los usuarios, solicita el ID, valida el acceso y registra el resultado en el archivo de auditoría.

```

34      if usuario:
35          mensaje = f"ACCESO PERMITIDO - {usuario['nombre_empleado']}"
36          print("CERRADURA ABIERTA por 5 segundos")
37      else:
38          mensaje = "ALERTA - ACCESO DENEGADO"
39          print("ALARMA ACTIVADA - Intento de acceso no autorizado")
40
41      registrar_log(mensaje)
42
43  if __name__ == "__main__":
44      main()

```

Se registra el mensaje según el ID si es valido se registrará en nombre del empleado así como se abrirá la cerradura y si es invalido se negara el acceso y se activará una alarma.

if name == "main"= permite que el programa ejecute la función main() automáticamente cuando se corre el archivo.

index.php

El HTML define la estructura visual del monitor de seguridad, comenzando con la declaración del documento y el idioma, seguido de la sección <head> donde se configuran los metadatos, el título de la página y los estilos CSS que dan el diseño rosado, incluyendo fondo, tabla y colores para los estados. En el <body> se muestra un título centrado y una tabla que organiza los registros de auditoría, donde cada fila representa un intento de acceso. Esta estructura permite que el contenido generado dinámicamente por PHP se integre dentro de la tabla, logrando una interfaz clara, ordenada y fácil de interpretar para el usuario.

index.php

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <title>Monitor de Seguridad</title>
6      <style>
7          body {
8              font-family: Arial;
9              background: linear-gradient(135deg, #ffe6f0, #ffc0cb);
10             text-align: center;
11         }
12         h2 {
13             color: #ec5ca4;
14         }
15         table {
16             margin: auto;
17             border-collapse: collapse;
18             width: 80%;
19             background: #fff0f5;
20             border-radius: 10px;
21             overflow: hidden;
22             box-shadow: 0px 4px 10px rgba(0,0,0,0.1);
23         }
24         th, td {
25             padding: 12px;
26             border-bottom: 1px solid #f8bbd0;
27         }
28         th {
29             background: #ff69b4;
30             color: white;
31         }
32         tr:hover {
33             background: #ffe4ec;
34         }
35         /* Acceso permitido */
36         .permitido {
37             background-color: #d4edda;
38             color: #50bd69;
39             font-weight: bold;
40         }
41         /* Acceso denegado */
42         .denegado {
43             background-color: #f8d7da;
44             color: #f74a5b;
45             font-weight: bold;
46         }
47     </style>
48 </head>
49 <body>
50     <h2>Monitor de Seguridad </h2>
51     <table>
52     <tr>
53         <th>Registro</th>
54     </tr>
```

```
55 <?php
56 $archivo = "auditoria.txt";
57
```

\$archivo = "auditoria.txt" se define el nombre del archivo que contiene los registros.

```
58 if (file_exists($archivo)) {
59     $lineas = file($archivo);
60     foreach ($lineas as $linea) {
61         $clase = "";
```

if = (file_exists(\$archivo)) Verifica si el archivo existe para evitar errores.

\$lineas = file(\$archivo) lee todo el archivo y guarda cada línea en un arreglo.

foreach = (\$lineas as \$linea) Recorre cada registro del archivo uno por uno.

\$clase = variable donde se guardará la clase CSS (color de la fila).

```
62         if (strpos($linea, "PERMITIDO") !== false) {
63             $clase = "permitido";
64         }
65
66         if (strpos($linea, "ALERTA") !== false || strpos($linea, "DENEGADO") !== false) {
67             $clase = "denegado";
68         }
```

si la línea contiene "PERMITIDO" se asigna la clase verde.

si contiene "ALERTA" o "DENEGADO" se asigna la clase roja.

```
70         echo "<tr class='$clase'><td>$linea</td></tr>";
71     }
```

muestra cada línea en una fila de la tabla con su color correspondiente.

```
72 } else {
73     echo "<tr><td>No hay registros disponibles</td></tr>";
74 }
75 ?>
```

si el archivo no existe muestra un mensaje en la tabla.

Libreria

usuarios.json

El archivo JSON se utiliza para almacenar la información de los usuarios de manera organizada y estructurada en forma de objetos y listas, donde cada usuario contiene datos como su ID de tarjeta, nombre, departamento y nivel de seguridad. Esta estructura jerárquica permite que el sistema acceda fácilmente a los datos mediante claves, facilitando la búsqueda y validación de usuarios.

```
{ } usuarios.json > [ ] usuarios > { } 2 > abc nombre_empleado
1  {
2    "usuarios": [
3      {
4        "id_tarjeta": "1031",
5        "nombre_empleado": "Lucia Martinez",
6        "departamento": "Sistemas",
7        "nivel_seguridad": 3
8      },
9      {
10       "id_tarjeta": "1032",
11       "nombre_empleado": "Jacqueline Sanchez",
12       "departamento": "Jefa de planta",
13       "nivel_seguridad": 3
14     },
15     {
16       "id_tarjeta": "1033",
17       "nombre_empleado": "Carlos Ramirez",
18       "departamento": "Ingeniero quimico",
19       "nivel_seguridad": 3
20     },
21     {
22       "id_tarjeta": "1021",
23       "nombre_empleado": "Maria Cardona",
24       "departamento": "Tecnica de laboratorio",
25       "nivel_seguridad": 2
26     },
27     {
28       "id_tarjeta": "1011",
29       "nombre_empleado": "Andres Gonzalez",
30       "departamento": "Auxiliar de almacen",
31       "nivel_seguridad": 1

```

Jerarquía de JSON

La jerarquía del archivo JSON se diseñó utilizando una estructura de tipo objeto principal que contiene un arreglo llamado "usuarios", donde cada elemento representa a un empleado con sus atributos como id de tarjeta,

nombre, departamento y nivel de seguridad. Esta organización permite acceder fácilmente a cada usuario como un objeto independiente, facilitando la búsqueda y validación dentro del sistema.

Relación entre JSON y auditoría.txt

La relación entre el JSON y el log de auditoría se da en el momento en que el programa en Python consulta el JSON para verificar si el ID ingresado existe; dependiendo del resultado, se genera un registro en el archivo TXT. Es decir, el JSON actúa como la base de datos de usuarios autorizados, mientras que el archivo de auditoría guarda el historial de todos los intentos de acceso, conectando así la validación con el registro de eventos.

Registro de actividad

auditoría.txt

El archivo TXT se utiliza para registrar todos los intentos de acceso al sistema en forma de texto plano, donde cada línea representa un evento con su fecha, hora y resultado (permitido o denegado). Este tipo de archivo es ideal porque es simple, ligero y rápido de manejar, permitiendo agregar información fácilmente sin sobrescribir datos anteriores.

```
≡ auditoria.txt
1  2026-04-14 10:15:32 - ACCESO PERMITIDO - Lucia Martinez
2  2026-04-14 10:17:05 - ALERTA - ACCESO DENEGADO
3  2026-04-14 10:20:11 - ACCESO PERMITIDO - Jacqueline Sanchez
4  2026-04-14 10:22:48 - ALERTA - ACCESO DENEGADO
5  2026-04-14 22:03:20 - ACCESO PERMITIDO - Carlos Ramirez
6  2026-04-14 22:47:57 - ACCESO PERMITIDO - Lucia Martinez
7  2026-04-14 22:48:51 - ACCESO PERMITIDO - Lucia Martinez
8  2026-04-14 22:49:42 - ALERTA - ACCESO DENEGADO
```

Monitor de Seguridad	
Registro	
2026-04-14 10:15:32	ACCESO PERMITIDO - Lucia Martinez
2026-04-14 10:17:05	ALERTA - ACCESO DENEGADO
2026-04-14 10:20:11	ACCESO PERMITIDO - Jacqueline Sanchez
2026-04-14 10:22:48	ALERTA - ACCESO DENEGADO
2026-04-14 22:03:20	ACCESO PERMITIDO - Carlos Ramirez
2026-04-14 22:47:57	ACCESO PERMITIDO - Lucia Martinez
2026-04-14 22:48:51	ACCESO PERMITIDO - Lucia Martinez
2026-04-14 22:49:42	ALERTA - ACCESO DENEGADO

El monitor de seguridad es una interfaz web desarrollada en PHP que muestra en tiempo real los registros de acceso almacenados en el archivo auditoria.txt. El sistema lee cada intento de acceso generado por el script en Python y lo presenta en una tabla organizada donde se diferencian visualmente los accesos permitidos en color verde y los accesos denegados en color rojo. Gracias a esta visualización el usuario puede identificar rápidamente eventos importantes o intentos no autorizados. En conjunto el monitor funciona como una herramienta de supervisión sencilla pero efectiva que permite llevar un control claro y continuo de la seguridad dentro del sistema.

Resultado del ingreso de ID

Aceptado

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS C:\xampp\htdocs\PROYECTO ADMIN> & C:/Users/.../troll.py
Ingrese ID de tarjeta: 1031
CERRADURA ABIERTA por 5 segundos
PS C:\xampp\htdocs\PROYECTO ADMIN> |
```

Denegado

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\xampp\htdocs\PROYECTO ADMIN> & C:/Users/lucia
trol.py"
Ingrese ID de tarjeta: 1627
ALARMA ACTIVADA - Intento de acceso no autorizado
PS C:\xampp\htdocs\PROYECTO ADMIN> |
```

archivos ('r', 'a')

El equipo utilizó diferentes modos de apertura de archivos según la función de cada uno. El archivo usuarios.json se abre en modo 'r' (lectura) porque únicamente se necesita consultar la información sin modificarla, lo que garantiza la integridad de los datos. Por otro lado, el archivo auditoria.txt se abre en modo 'a' (append) para agregar nuevos registros sin borrar los anteriores, permitiendo mantener un historial completo de los accesos. Además, se implementó manejo de errores mediante excepciones para detectar problemas como archivos inexistentes o JSON mal formado, evitando que el sistema falle y mostrando mensajes claros al usuario, lo que hace que el sistema sea más seguro y confiable.`

Conclusión

En conclusión, se desarrolló un sistema de control de acceso funcional y eficiente utilizando herramientas simples como JSON, TXT, Python y PHP. El sistema permite validar usuarios, registrar intentos de acceso y visualizar la información de manera clara, cumpliendo con los requerimientos de seguridad en un entorno industrial. Además, el uso de archivos planos demostró ser una solución práctica, rápida y confiable, ideal para sistemas que no requieren infraestructuras complejas, garantizando así un buen desempeño y facilidad de implementación.